WEEK 5 – PAYMENT TRANSACTION MANAGEMENT SYSTEM

1. Scenario

The Art Work Shop database includes a Payment Transaction Module to store and manage customer payment details. It records the payment ID, order ID, payment date, payment method, payment status, and transaction amount. This module helps track payments and analyse transactions made for customer orders.

2. Create Payment Table

```
SQL> CREATE TABLE Payment (Payment_ID NUMBER, Order_ID NUMBER, Payment_Date
DATE, Payment_Mode VARCHAR2(20), Payment_Status VARCHAR2(20), Transaction_Am
ount NUMBER(10,2));

Table created.
```

3. Applying Constraints

- **PRIMARY KEY:** Uniquely identifies each payment record using Payment_ID.
- **FOREIGN KEY:** Links Order_ID in the Payment table with Order_ID in the Orders table.
- **NOT NULL:** Ensures that required payment fields cannot be left empty.
- **CHECK:** Ensures that the payment amount is greater than zero.
- **UNIQUE:** Ensures that each order has only one payment record.

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT CHK_PAYMENT_AMOUNT
  3  CHECK (Payment_Amount > 0);

Table altered.
```


```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT CHK_PAYMENT_AMOUNT
  3  CHECK (Payment_Amount > 0);

Table altered.
```

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT CHK_PAYMENT_METHOD
  3  CHECK (Payment_Method IN ('UPI', 'Card', 'Cash', 'Net Banking'));

Table altered.

SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT CHK_PAYMENT_STATUS
  3  CHECK (Payment_Status IN ('Pending', 'Completed', 'Failed'));

Table altered.
```

4. Establishing Relationship

One-to-One (1:1) Relationship: The Orders table is linked to the Payment table using Order_ID as a Foreign Key and UNIQUE constraint, ensuring that each order has only one payment record.

5. Inserting Values into Payment Table

```
SQL> INSERT INTO Payment
  2  VALUES (4, 1004, DATE '2026-10-04', 18000.00, 'UPI', 'Success');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (5, 1005, DATE '2026-10-05', 4500.00, 'Cash', 'Pending');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (6, 1006, DATE '2026-10-06', 7200.00, 'Card', 'Success');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (7, 1007, DATE '2026-10-07', 6200.00, 'Net Banking', 'Failed');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (8, 1008, DATE '2026-10-08', 9500.00, 'UPI', 'Pending');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (9, 1009, DATE '2026-10-09', 11000.00, 'Card', 'Success');

1 row created.

SQL> INSERT INTO Payment
  2  VALUES (10, 1010, DATE '2026-10-10', 15000.00, 'Cash', 'Failed');

1 row created.
```

```
SQL> SELECT * FROM Payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_AMOUNT PAYMENT_METHOD
---------- ---------- --------- -------------- --------------------
PAYMENT_STATUS
--------------------
         1       1001 01-OCT-26           5000 UPI
Success

         2       1002 02-OCT-26           3500 Card
Pending

         3       1003 03-OCT-26           8400 Net Banking
Failed


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_AMOUNT PAYMENT_METHOD
---------- ---------- --------- -------------- --------------------
PAYMENT_STATUS
--------------------
         4       1004 04-OCT-26          18000 UPI
Success

         5       1005 05-OCT-26           4500 Cash
Pending

         6       1006 06-OCT-26           7200 Card
Success


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_AMOUNT PAYMENT_METHOD
---------- ---------- --------- -------------- --------------------
PAYMENT_STATUS
--------------------
         7       1007 07-OCT-26           6200 Net Banking
Failed

         8       1008 08-OCT-26           9500 UPI
Pending

         9       1009 09-OCT-26          11000 Card
```

```
         8       1009 28-SEP-26 Cash                Failed
              3400

         9       1010 29-SEP-26 UPI                 Pending
              7500


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        10       1011 30-SEP-26 UPI                 Success
             60000


10 rows selected.
```

```
PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_AMOUNT PAYMENT_METHOD
---------- ---------- --------- -------------- --------------------
PAYMENT_STATUS
--------------------
         7       1007 07-OCT-26           6200 Net Banking
Failed

         8       1008 08-OCT-26           9500 UPI
Pending

         9       1009 09-OCT-26          11000 Card
Success


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_AMOUNT PAYMENT_METHOD
---------- ---------- --------- -------------- --------------------
PAYMENT_STATUS
--------------------
        10       1010 10-OCT-26          15000 Cash
Failed


10 rows selected.
```

6. Update and Delete Operations

Update Payment Amount

```
SQL> UPDATE Payment
  2  SET Payment_Amount = 5000.00
  3  WHERE Payment_ID = 5;

1 row updated.
```

Delete a Payment Record

```
SQL> DELETE FROM Payment
  2  WHERE Payment_ID = 10;

1 row deleted.
```

7. Manage Successful and Failed Payments

7.1 Display All Successful Payments

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Success';
```

7.2 Find Failed Transactions

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';
```

7.3 Count Total Successful and Failed Payments

```
SQL> SELECT Payment_Status, COUNT(*) FROM Payment WHERE Payment_Status IN ('
Success','Failed') GROUP BY Payment_Status;
```

7.4 Update Failed Payments After Retry

```
SQL> UPDATE Payment SET Payment_Status = 'Success' WHERE Payment_ID = 7;

1 row updated.
```

7.5 Identify Pending Transactions

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Pending';
```

8. Analyse Payment Methods Used by Customers

Report 1: Payment Mode Analysis

a) Number of Transactions Using UPI

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM Payment WHERE Payment_Method =
 'UPI';

UPI_TRANSACTIONS
----------------
               3
```

b) Number of Card Payments

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM Payment WHERE Payment_Method = 'C
ard';

CARD_PAYMENTS
-------------
            3
```

c) Most Preferred Payment Method

```
SQL> SELECT Payment_Method, COUNT(*) AS TOTAL FROM Payment GROUP BY Payment_
Method ORDER BY TOTAL DESC;

PAYMENT_METHOD            TOTAL
-------------------- ----------
Card                          3
UPI                           3
Net Banking                   2
Cash                          1
```

The payment method with the highest transaction count is the most preferred payment method.

Report 2: Revenue Analysis

a) Total Revenue Generated

```
SQL> SELECT SUM(Payment_Amount) AS TOTAL_REVENUE FROM Payment;

TOTAL_REVENUE
-------------
        73800
```

b) b) Revenue by Payment Method

```
SQL> SELECT Payment_Method, SUM(Payment_Amount) AS REVENUE FROM Payment GROU
P BY Payment_Method;

PAYMENT_METHOD          REVENUE
-------------------- ----------
Net Banking               14600
Card                      21700
Cash                       5000
UPI                       32500
```

c) c) Average Transaction Amount

```
SQL> SELECT AVG(Payment_Amount) AS AVERAGE_TRANSACTION_AMOUNT FROM Payment;

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                      8200
```

Report 3: Customer Payment History

Aim: To display customer name, order ID, payment method, amount, and payment status.

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name, O.Order_ID, P.Payment_Method, P.Payment_Amount, P.Payment_Status FROM Customer C JOIN Order
s O ON C.Customer_ID = O.Customer_ID JOIN Payment P ON O.Order_ID = P.Order_ID;

CUSTOMER_NAME                          ORDER_ID PAYMENT_METHOD
-------------------------------------- ---------- --------------------
PAYMENT_AMOUNT PAYMENT_STATUS
-------------- --------------------
Ricky Ponting                                1001 UPI
         5000 Success

Ricky Ponting                                1002 Card
         3500 Pending

Ricky Ponting                                1003 Net Banking
         8400 Failed


CUSTOMER_NAME                          ORDER_ID PAYMENT_METHOD
-------------------------------------- ---------- --------------------
PAYMENT_AMOUNT PAYMENT_STATUS
-------------- --------------------
Ricky Ponting                                1004 UPI
        18000 Success

Ricky Ponting                                1005 Cash
         5000 Pending

Ricky Ponting                                1006 Card
         7200 Success


CUSTOMER_NAME                          ORDER_ID PAYMENT_METHOD
-------------------------------------- ---------- --------------------
PAYMENT_AMOUNT PAYMENT_STATUS
-------------- --------------------
Ricky Ponting                                1007 Net Banking
         6200 Success

Ricky Ponting                                1008 UPI
         9500 Pending
```

9. Conclusion

The Payment Transaction Management System stores and manages customer payment details. Primary key, foreign key, NOT NULL, CHECK, and UNIQUE constraints maintain data integrity. SQL queries are used to analyse payment methods, calculate revenue, manage payment statuses, and display customer payment history.